



ENTREGA DE TALLER

Realizado por:

Andres Centanaro

Jose Jaramillo

Santiago Botero

TABLA DE CONTENIDO

1- Estilo: MVC... etc

- Definicion
- Caracterisiticas
- Historia y evolucion
- Pros/Cons
- Derivados
- Casos de Uso

3- Backend: Express

- Definicion
- Caracterisiticas
- Historia
- Pros/Cons
- Casos de Uso

2- Frontend: Vuejs 3

- Definicion
- Caracterisiticas
- Historia
- Pros/Cons
- Casos de Uso

4- Intregación: GraphQL

- Definicion
- Caracterisiticas
- Historia
- Pros/Cons
- Casos de Uso

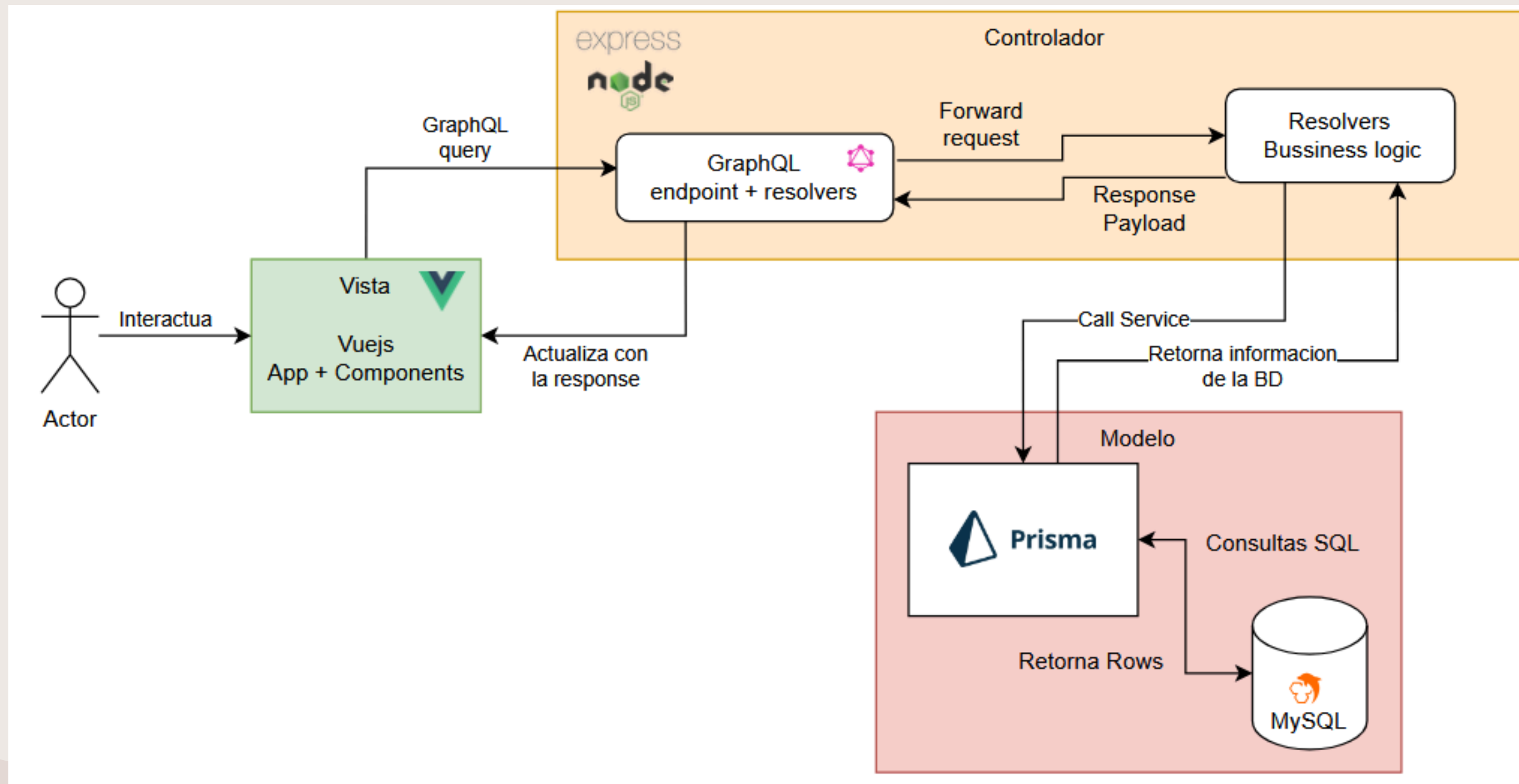
5- Análisis del stack

- Popularidad del stack
- Mercado Laboral
- Matrices de análisis
 - Principios SOLID
 - Atributos de calidad
 - Tácticas
 - Patrones

6- Ejemplo y Diagramas

- Alto Nivel
- C4
- Despliegue
- Paquetes UML
- Muestra de funcionalidad

ESTILO DE DISEÑO MVC



ESTILO DE DISEÑO

MVC

Definición

Patrón de arquitectura de software que separa la aplicación en tres componentes interconectados:

Modelo (datos y lógica de negocio)

Vista (interfaz de usuario)

Controlador (intermediario que gestiona la comunicación).



ESTILO DE DISEÑO

MVC

Características

- Separación de responsabilidades clara entre componentes
- Bajo acoplamiento entre capas
- Alta cohesión dentro de cada componente
- Reutilización de código
- Desarrollo paralelo entre equipos
- Facilita pruebas unitarias

ESTILO DE DISEÑO

MVC

Historia & Evolución

- **1979:** Creado por Trygve Reenskaug en Xerox PARC para Smalltalk
- **1990s:** Adoptado en aplicaciones de escritorio
- **2000s:** Popularizado con frameworks web (Ruby on Rails, ASP.NET MVC)
- **Actualidad:** Evolución a patrones derivados (MVVM, MVP, MVT)



ESTILO DE DISEÑO

MVC

Patrones Derivados

(MVVM, MVP, MVT)

PROS

- Código organizado y mantenible
- Facilita trabajo en equipo
- Permite múltiples vistas del mismo modelo
- Simplifica debugging y testing

CONS

- Curva de aprendizaje inicial
- Puede ser excesivo para proyectos pequeños
- Mayor complejidad en la estructura
- Posible *overhead* en aplicaciones simples

CASOS DE USO

MVC

- Aplicaciones web con interfaces dinámicas
- Sistemas con múltiples representaciones de datos
- Proyectos con equipos separados (frontend/backend)
- Aplicaciones que requieren escalabilidad
- Sistemas donde la lógica de negocio cambia frecuentemente

CASOS DE ÉXITO

MVC

- Ruby on Rails: Basecamp, GitHub, Shopify
- Django (MVT): Instagram, Pinterest, Mozilla
- ASP.NET MVC: Stack Overflow, Microsoft
- Spring MVC: LinkedIn, Netflix
(microservicios)
- Laravel: Pfizer, BBC, 9GAG

ESTILO DE DISEÑO

VUE.JS

Definición

Framework de JavaScript para construir interfaces de usuario interactivas.

Se enfoca en la capa de **vista** y puede integrarse fácilmente en proyectos existentes o utilizarse para aplicaciones complejas de página única.

ESTILO DE DISEÑO

VUE.JS

Características

- Sistema de reactividad automático
- Progresivo - Se puede adoptar incrementalmente
- Componentes reutilizables con templates
- Virtual DOM para renderizado eficiente
- Directivas (v-if, v-for, v-model)
- Sistema de routing y gestión de estado integrados
- Curva de aprendizaje suave

ESTILO DE DISEÑO

VUE.JS

Historia & Evolución

- **2014:** Creado por Evan You (ex-Google)
- **2015:** Vue.js 1.0 - Adopción inicial
- **2016:** Vue.js 2.0 - Virtual DOM, renderizado server-side
- **2020:** Vue.js 3.0 - Composition API, mejor performance
- **Actualidad:** Uno de los 3 frameworks más populares (con React y Angular)

PROS

- Fácil de aprender y usar
- Documentación excelente y en español
- Flexibilidad y escalabilidad
- Rendimiento optimizado
- Comunidad activa y creciente
- Ecosistema completo (Vuex, Vue Router, Nuxt.js)

CONS

- Menos recursos corporativos (comparado con Angular/React)
- Ecosistema de plugins y templates más limitado
- Menos oferta laboral en algunos mercados

CASOS DE USO

VUE.JS

- Aplicaciones web de página única (SPA)
- Interfaces de usuario dinámicas y reactivas
- Dashboards y paneles administrativos
- Prototipos rápidos y MVPs
- Migración progresiva de aplicaciones legacy
- Progressive Web Apps (PWA)
- Aplicaciones móviles (con Ionic/Capacitor)

CASOS DE ÉXITO

VUE.JS

- Alibaba: Plataforma e-commerce líder mundial
- Xiaomi: Sitio web corporativo
- Nintendo: Portal web europeo
- Grammarly: Editor de escritura
- GitLab: Interfaz de usuario
- Adobe Portfolio: Creador de portfolios
- Laravel Ecosystem: Nova, Spark, Forge

ESTILO DE DISEÑO

EXPRESS

Definición

Framework web minimalista y flexible para Node.js que proporciona un conjunto completo de características para aplicaciones web y móviles.

Facilita la creación de APIs y servidores web con JavaScript o Typescript en el backend.

ESTILO DE DISEÑO **EXPRESS**

Características

- Minimalista - Máxima flexibilidad
- Middleware - Sistema de funciones encadenables
- Routing robusto - Manejo de rutas y métodos HTTP
- Gestión de peticiones/respuestas simplificada
- Manejo de archivos estáticos

ESTILO DE DISEÑO **EXPRESS**

Historia & Evolución

- **2010:** Creado por TJ Holowaychuk
- **2014:** Adquirido por StrongLoop, luego por IBM
- **2015:** Express 4.0 - Independización del middleware
- **2016:** Pasa a la Node.js Foundation
- **Actualidad:** Framework backend más popular de Node.js, se considera el estándar de facto

PROS

- Rápido y ligero
- Curva de aprendizaje sencilla
- Gran ecosistema de middleware
- Comunidad masiva y madura
- Documentación extensa
- Flexibilidad total en arquitectura

CONS

- Requiere configuración manual
- Sin estructura predefinida (puede causar desorden)
- Manejo de código asíncrono complejo
- Sin características avanzadas incluidas (ORM, validación)

CASOS DE USO

EXPRESS

- APIs RESTful y servicios web
- Aplicaciones backend de SPAs
- Microservicios
- Servidores proxy y middleware
- Aplicaciones en tiempo real (con Socket.io)
- Prototipos y MVPs rápidos
- Aplicaciones empresariales escalables
- Backends para aplicaciones móviles

CASOS DE ÉXITO

EXPRESS

- Uber: APIs y microservicios
- IBM: Diversas aplicaciones empresariales
- Accenture: Soluciones web corporativas
- Fox Sports: Plataforma de streaming
- PayPal: Migración de Java a Node.js/Express
- Netflix: Varias herramientas internas
- MySpace: Refactorización completa del backend
- Trello: Backend de la aplicación

ESTILO DE DISEÑO

GRAPHQL

Definición

Lenguaje de consulta y manipulación de datos para APIs, desarrollado como alternativa a REST.

Permite a los clientes solicitar exactamente los datos que necesitan en una sola petición, evitando over-fetching y under-fetching.



ESTILO DE DISEÑO

GRAPHQL

Características

- Consultas declarativas - El cliente define la estructura de respuesta
- Un solo endpoint - No múltiples URLs como REST
- Tipado fuerte - Schema definido y validado
- Resolvers - Funciones que obtienen los datos
- Subscripciones - Actualizaciones en tiempo real



ESTILO DE DISEÑO

GRAPHQL

Historia & Evolución

- **2012:** Creado internamente en Facebook
- **2015:** Liberado como open source
- **2016:** Adopción por GitHub, Pinterest, Shopify
- **2018:** Transferido a GraphQL Foundation (Linux Foundation)
- **Actualidad:** Estándar adoptado por grandes empresas, con un ecosistema maduro

PROS

- Eficiencia en transferencia de datos
- Reducción de peticiones HTTP
- Documentación automática
- Tooling excelente (GraphQL, Playground)
- Facilita desarrollo paralelo
- Mejor experiencia de desarrollo

CONS

- Curva de aprendizaje pronunciada
- Caché más complejo que REST
- Queries complejas pueden afectar performance
- Overhead para APIs simples
- Dificulta rate limiting por endpoint



CASOS DE USO

GRAPHQL

- Dashboards con necesidades de datos variables
- Aplicaciones con datos relacionales complejos
- Productos con actualizaciones en tiempo real
- Plataformas donde el ancho de banda es limitado
- APIs públicas con diversos consumidores

CASOS DE ÉXITO

GRAPHQL

- Airbnb: Plataforma completa
- Netflix: Federated GraphQL Gateway
- PayPal: Checkout y transacciones
- The New York Times: CMS y aplicaciones

POPULARIDAD FRONTEND



REACT

VUEJS

ANGULAR

SVELTE

SOLID

POPULARIDAD BACKEND

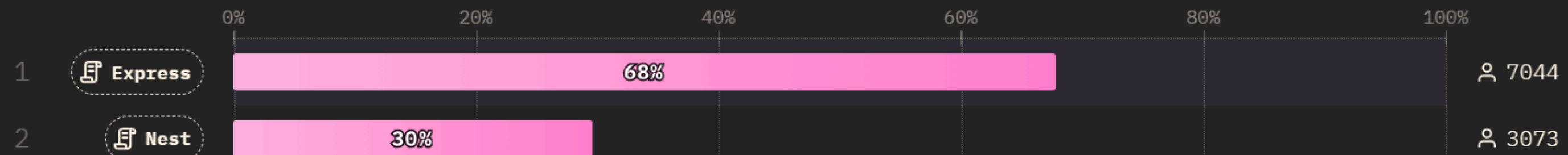
BACK-END FRAMEWORKS

21

Frameworks for generating APIs and managing back-ends

All Respondents

Query Builder



PRINCIPIOS SOLID

Principio SOLID

MVC

Vue.js

Express.js

GraphQL

S - Single Responsibility

✓✓✓ Cada capa tiene responsabilidad única

✓✓ Componentes con una función específica

✓ Middleware con funciones específicas

✓✓ Resolvers separados por tipo

O - Open/Closed

✓✓ Extensible sin modificar código base

✓✓✓ Mixins, plugins, composables

✓✓✓ Sistema de middleware extensible

✓✓ Schema extensible sin romper clientes

L - Liskov Substitution

✓✓ Controladores intercambiables

✓ Componentes intercambiables

⚠ Depende de implementación

✓ Tipos abstractos e interfaces

I - Interface Segregation

✓✓ Interfaces específicas por capa

✓ Props y eventos específicos

⚠ No forzado por el framework

✓✓✓ Fragmentos y tipos específicos

D - Dependency Inversion

✓✓✓ Modelo independiente de Vista

✓✓ Inyección de dependencias

✓ Middleware inyectable

✓✓ Resolvers desacoplados de fuentes

ATRIBUTOS CALIDAD

Atributo de Calidad	MVC	Vue.js	Express.js	GraphQL
Performance	✓✓ Buena separación optimiza	✓✓✓ Virtual DOM, reactividad	✓✓✓ Asíncrono, no bloqueante	✓✓ Eficiente pero requiere optimización
Escalabilidad	✓✓✓ Horizontal y vertical	✓✓ Buena con Vuex/Pinia	✓✓✓ Microservicios friendly	✓✓✓ Excelente con federación
Mantenibilidad	✓✓✓ Código organizado	✓✓✓ Componentes reutilizables	✓✓ Depende de arquitectura	✓✓ Schema como contrato
Testabilidad	✓✓✓ Capas independientes	✓✓✓ Testing utilities incluidas	✓✓ Fácil de testear	✓✓✓ Mocking sencillo
Seguridad	✓✓ Separación reduce riesgos	✓ XSS protection básica	✓✓ Middleware de seguridad	⚠ Requiere implementación cuidadosa
Usabilidad	✓✓ UX/UI desacoplado	✓✓✓ Interfaces reactivas	N/A Backend	✓✓✓ Cliente obtiene lo que necesita
Disponibilidad	✓✓ Fácil replicación	✓✓ SSR con Nuxt	✓✓✓ Clustering, PM2	✓✓ Caché complejo pero efectivo
Interoperabilidad	✓✓ Estándar conocido	✓✓ APIs REST/GraphQL	✓✓✓ Compatible con todo	✓✓✓ Agnós

TÁCTICAS

Táctica	MVC	Vue.js	Express.js	GraphQL
Caché	✓✓ Cache en Modelo/Vista	✓✓ Computed properties	✓✓ Middleware de caché	⚠️✓ Complejo, requiere normalización
Load Balancing	✓✓ Aplicable en servidor	✓ SSR con balanceador	✓✓✓ Nginx, cluster mode	✓✓ API Gateway
Replicación	✓✓✓ Múltiples instancias	✓✓ Hydration de SSR	✓✓✓ Stateless scaling	✓✓ Schema stitching
Monitoreo	✓✓ Logs por capa	✓ Vue DevTools	✓✓✓ Morgan, Winston	✓✓✓ Apollo Studio, tracing
Validación	✓✓✓ En Modelo y Controlador	✓✓ Vuelidate, VeeValidate	✓✓ express-validator	✓✓✓ Schema validation automática
Autenticación	✓✓✓ En Controlador	✓✓ Vue Router guards	✓✓✓ Passport.js, JWT	✓✓ Context y middleware
Throttling	✓✓ Implementación manual	✓ Debounce en eventos	✓✓✓ express-rate-limit	⚠️✓ Complexity y depth limiting
Retry/Fallback	✓ Por implementación	✓✓ Axios interceptors	✓✓ Middleware de errores	✓✓ Apollo retry policies

PATRONES DISEÑO

Patrón de Diseño	MVC	Vue.js	Express.js	GraphQL
MVC/MVVM	✓✓✓ Es el patrón	✓✓✓ MVVM nativo	⚠ Solo Controller	⚠ Resolver pattern
Observer	✓✓✓ Vista observa Modelo	✓✓✓ Sistema de reactividad	✓ Event Emitters	✓✓ Subscriptions
Strategy	✓✓ Controladores alternos	✓✓ Composables	✓✓✓ Middleware intercambiable	✓✓ Custom resolvers
Factory	✓✓ Creación de objetos	✓✓ Component factories	✓ Router factories	✓ Type factories
Singleton	✓✓ Modelo único	✓✓✓ Vuex/Pinia stores	✓✓ App instance	✓ Schema singleton
Decorator	✓ Extensión de clases	✓ Mixins, Directives	✓✓✓ Middleware chain	✓✓ Field resolvers
Repository	✓✓✓ Acceso a datos en Modelo	✓ Services layer	✓✓ Data access layer	✓✓✓ Data sources
Dependency Injection	✓✓ Inyección en Controlador	✓✓ provide/inject	✓ Constructor injection	✓✓ Context injection

MERCADO LABORAL

Aspecto Laboral	MVC	Vue.js	Express.js	GraphQL
Demanda Global	✓✓✓ Alta (patrón universal)	✓✓✓ Alta y creciente	✓✓✓ Muy alta	✓✓ Media-Alta, creciente
Salario Promedio	Variable por framework	60K-60K-110K USD	70K-70K-120K USD	80K-80K-130K USD (premium)
Curva de Aprendizaje	✓✓ Media	✓✓✓ Fácil	✓✓ Fácil-Media	⚠ Compleja
Ofertas Junior	✓✓✓ Muchas	✓✓✓ Abundantes	✓✓✓ Abundantes	✓ Pocas (requiere experiencia)
Ofertas Senior	✓✓✓ Muchas	✓✓✓ Muchas	✓✓✓ Muchas	✓✓✓ Muchas y bien pagadas
Empresas que Usan	Todas	Alibaba, Xiaomi, GitLab	Uber, PayPal, Netflix	GitHub, Shopify, Airbnb
Stack Común	Spring, Django, Laravel	MEVN, Nuxt, Quasar	MERN, MEAN, MEVN	Apollo, Prisma, Hasura
Freelance	✓✓✓ Muy demandado	✓✓✓ Muy demandado	✓✓✓ Muy demandado	✓✓ Demandado (nicho)
Futuro/Tendencia	✓✓✓ Estable, fundamental	✓✓✓ Crecimiento sostenido	✓✓✓ Estable, maduro	✓✓✓ Crecimiento acelerado
Certificaciones	Varias por framework	✓ Vue.js Certification	⚠ Pocas oficiales	✓ Apollo GraphQL Cert

NUESTROS VALORES

Elegancia

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Duis vulputate nulla at
ante rhoncus, vel efficitur
felis condimentum. Proin
odio odio.

Sostenibilidad

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Duis vulputate nulla at
ante rhoncus, vel efficitur
felis condimentum. Proin
odio odio.

Calidad

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Duis vulputate nulla at
ante rhoncus, vel efficitur
felis condimentum. Proin
odio odio.

DIAGRAMAS C4

PLAN DE ACCIÓN

Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Duis vulputate nulla at ante rhoncus, vel efficitur felis
condimentum. Proin odio odio.



Investigación



Análisis



Diseño



Implementación



Ventas

RESUMEN DEL PROYECTO



Diseño de interiores

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vulputate nulla at ante rhoncus, vel efficitur felis condimentum. Proin odio odio.



Decoración

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vulputate nulla at ante rhoncus, vel efficitur felis condimentum. Proin odio odio.

EL EQUIPO



Alicia Ortiz
Gerente



Pablo Martín
Diseño de interiores



Carlos Méndez
Ilustrador



Rosa Martínez
Ventas



**MUCHAS
GRACIAS**

Página de Recursos

